

파티클 필터 & MCL 알고리즘

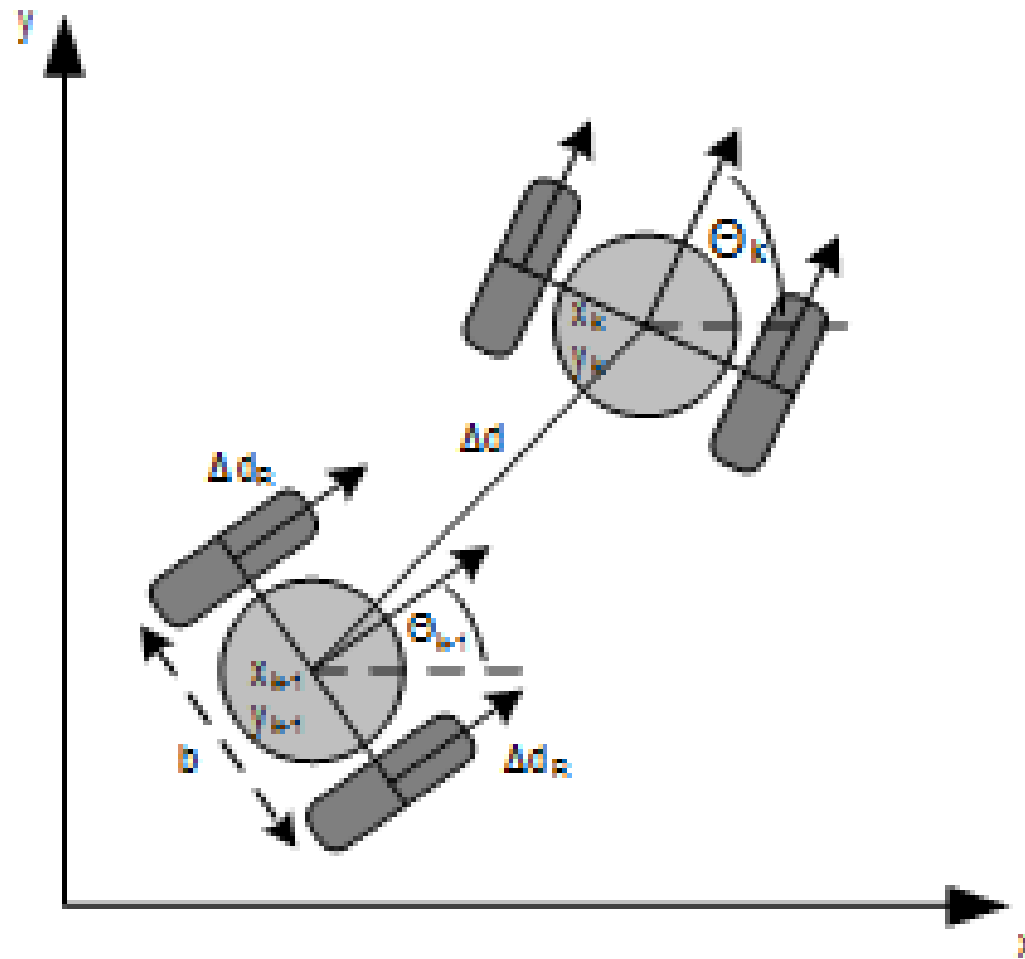
Particle Filter · MCL · AMCL

cchyun@gmail.com

Part 1. 로컬라이제이션(Localization) 개요

로컬라이제이션이 왜 필요한가?

- 로봇에게 정확한 지도가 있어도, 지도 위의 자신의 위치를 모르면 무용지물
 - "지도 속 나는 어디에 있나?"
 - 처음 켜진 로봇은 지도상의 자신의 위치(x , y , 방향 θ)를 모를 수 있음
 - 마치 낯선 건물 안에서 평면도 한 장만 들고 서 있는 것과 같음
 - 오도메트리(Odometry)의 한계
 - 바퀴 회전수·IMU로 "내가 얼마나 움직였나"를 계산하는 방식
 - 현실에서는 바퀴 미끄러짐(Wheel Slip), 센서 오차가 섞임
 - 처음엔 1cm 차이였던 오차가 시간이 지나면 수십 cm 이상으로 누적
 - 해결책: 센서 + 지도 비교로 누적 오차를 지속적으로 보정



- 로봇이 처한 상황에 따라 위치 추정의 난이도가 달라짐
 - ① 위치 추적 (Position Tracking) — 가장 쉬운 단계
 - 초기 위치를 정확히 아는 상태에서, 이동하는 로봇의 위치를 지속 업데이트
 - ② 글로벌 로컬라이제이션 (Global Localization) — 중간 난이도
 - 로봇이 지도의 어디에 있는지 **전혀 모르는** 상태에서 시작
 - 지도 전체에 "혹시 여기인가?" 하는 가설을 흘려야 함
 - ③ 납치된 로봇 문제 (Kidnapped Robot Problem) — 가장 어려운 단계
 - 잘 이동 중이던 로봇이 갑자기 **다른 곳으로 옮겨진 상황**
 - 자신이 생각한 위치가 완전히 틀렸음을 인지하고 위치를 처음부터 재탐색

Part 2. 파티클 필터 (Particle Filter)

- "어려운 수학 문제를 풀기 어렵다면, 수많은 가설을 던져서 맞히자"
 - 파티클(Particle, 입자)이란?
 - 지도 위에 흩뿌려진 수많은 '가상 로봇'들
 - 각 파티클은 "로봇이 여기에, 이 방향을 보고 있을 것" 이라는 하나의 가설
 - 가중치(Weight) 부여 과정
 - 로봇이 LiDAR로 주변 벽까지의 거리를 측정
 - 각 파티클 위치에서 "만약 진짜 로봇이 여기 있다면 무엇이 보였을까?"를 계산
 - 실제 센서 값과 비슷하면 높은 가중치, 다르면 낮은 가중치
 - 리샘플링(Resampling) — 생물 진화의 비유
 - 점수 낮은 파티클은 도태, 점수 높은 파티클은 복제·증식
 - "적자생존" 원리 → 가능성 높은 가설에 연산 자원을 집중

- MCL(Monte Carlo Localization): 파티클 필터를 로봇 위치 추정에 적용한 알고리즘
 - ① 예측 단계 (Motion Update)
 - 로봇이 1m 움직이면, 모든 파티클도 각자의 위치에서 1m 이동
 - 약간의 무작위 노이즈를 섞어 파티클을 흩어지게 함
 - → 오도메트리의 불확실성을 반영
 - ② 업데이트 단계 (Sensor Update)
 - 로봇이 LiDAR로 벽을 스캔
 - 지도와 스캔 데이터를 비교하여 각 파티클의 가중치(확률)를 계산
 - ③ 리샘플링 단계 (Resampling)
 - 높은 확률의 파티클들만 남기고, 그 주변에 새 파티클들을 채움
 - 이 루프 반복 → 파티클들이 실제 위치로 수렴(Convergence)

- MCL 루프 흐름

초기화

↓

지도 전체에 파티클을 무작위 배포

↓

[루프 시작]

① 예측: 로봇 이동량만큼 파티클 이동 + 노이즈 추가

↓

② 업데이트: LiDAR 스캔 vs 지도 비교 → 각 파티클 가중치 계산

↓

③ 리샘플링: 가중치 기반으로 파티클 재선택 (높은 확률 복제, 낮은 확률 제거)

[루프 반복]

↓

파티클 수렴 → 로봇 위치 확정

- 파티클 수는 정확도와 연산량의 균형이 중요
 - 파티클이 많을수록
 - 가설이 촘촘하여 위치를 더 정확하고 빠르게 찾음
 - 하지만 CPU가 각 파티클마다 가중치를 계산 → 연산 부담 증가
 - 너무 많으면 로봇의 제어 속도가 느려질 수 있음
 - 파티클이 적을수록
 - 연산량이 적어 빠른 처리 가능
 - 하지만 가설이 듥성듬성하여 위치 추정 정밀도 저하
 - 실용적 기준
 - 일반적으로 500 ~ 2000개 수준에서 운용
 - 로봇의 CPU 성능과 환경 복잡도에 맞게 조정

- Q. "왜 일부러 노이즈를 섞나요?"

- 로봇의 이동은 완벽하지 않음
- 파티클을 똑같은 위치에만 두면 실제 로봇이 미끄러졌을 때 추적에 실패
- 약간의 무작위성을 섞어야 예상치 못한 오차 상황에서도 로봇을 놓치지 않음

- Q. "리샘플링하면 가설이 다 똑같아지지 않나요?"

- 리샘플링은 가중치가 높은 지점 '**주변**'으로 다시 샘플링하는 것
- 이후 **예측 단계의 노이즈** 덕분에 다시 적절히 퍼지며 새로운 가설들을 생성
- 즉, 리샘플링 + 노이즈 추가의 조합이 다양성을 유지하는 핵심

Part 3. AMCL (Adaptive MCL)

- 기본 MCL은 파티클 수를 고정 / AMCL은 상황에 맞게 파티클 수를 자동 조절
 - KLD-Sampling 개요
 - KLD(Kullback-Leibler Divergence): 추정 분포와 실제 분포의 차이를 측정하는 척도
 - 위치를 확신할 때 (파티클들이 한곳에 수렴)
 - 파티클 수를 줄여서 CPU 자원을 절약
 - 위치를 잃었을 때 (가설이 사방으로 퍼짐)
 - 파티클 수를 대폭 늘려 로봇을 다시 찾을 확률을 높임
 - 결과: 효율성과 생존율을 동시에 확보

- AMCL이 위치를 추정하려면 3가지 입력이 필요
 - `/map` 토픽 (`nav_msgs/OccupancyGrid`)
 - `map_server`가 제공하는 정적 점유 그리드 지도
 - 없으면: 파티클 필터 초기화 불가 → 위치 추정 시작되지 않음
 - `/scan` 토픽 (`sensor_msgs/LaserScan`)
 - 2D LiDAR 실시간 거리 측정 데이터
 - 없으면: 가중치 계산 불가 → 파티클이 수렴하지 않고 계속 흩어짐
 - `/tf` (`odom → base_link` 변환)
 - 인코더·IMU 등으로 계산된 오도메트리 정보
 - 없으면: 예측 단계 불가 → 위치 추적 중단

- 추정된 위치 정보는 토픽과 TF 변환으로 발행
 - `/amcl_pose` 토픽 (`geometry_msgs/PoseWithCovarianceStamped`)
 - 지도 좌표계 상의 로봇 위치(x, y, θ)와 불확실성(Covariance) 포함
 - Nav2 플래너·컨트롤러가 경로 생성·추종에 활용
 - `/particle_cloud` 토픽 (`nav2_msgs/msg/ParticleCloud`)
 - 현재 로봇이 있을 법한 후보 위치 파티클들의 집합
 - RViz2에서 시각화 → 구름이 작고 밀집될수록 높은 정확도

- 전체 TF 흐름: `map → odom → base_link`
 - 오도메트리 (`odom → base_link`)
 - 바퀴 회전량 등을 이용해 **연속적이고 빠른** 상대 이동 거리 계산
 - 시간이 지날수록 오차 누적
 - AMCL (`map → odom`)
 - 지도를 기준으로 LiDAR 데이터를 비교
 - 오도메트리의 누적 오차를 보정하는 절대 위치 변환 계산
 - 역할 분담
 - 오도메트리: 로봇의 즉각적인 움직임 담당
 - AMCL: 그 움직임이 지도의 어느 좌표에 해당하는지 주기적으로 보정

- 로컬라이제이션 시작 시 로봇의 대략적인 위치를 알려주는 단계
 - 방법 ① RViz2 사용
 - 상단 툴바의 '2D Pose Estimate' 버튼 클릭
 - 지도상에서 로봇의 실제 위치와 방향을 마우스로 클릭 후 드래그
 - 방법 ② `/initialpose` 토픽 (코드·CLI에서 직접 설정)

```
ros2 topic pub --once /initialpose \  
  geometry_msgs/msg/PoseWithCovarianceStamped \  
  "{header: {frame_id: 'map'}, pose: {pose: {position: \  
  {x: 0.0, y: 0.0, z: 0.0}, orientation: {w: 1.0}}}}"
```



- **납치된 로봇 상황: 기존 파티클이 쓸모없어질 때 사용**

- 서비스 호출 방법

```
ros2 service call /global_localization std_srvs/srv/Empty {}
```

- **작동 원리**

- 호출 즉시 파티클들이 **지도의 자유 공간(Free space) 전체**에 재배포
- 이후 로봇을 제자리에서 회전 → LiDAR가 지형 특징과 일치하는 파티클의 가중치 상승
- → 위치를 다시 찾아감

- **언제 사용하나?**

- 로봇이 손으로 들어 옮겨진 경우
- 파티클이 완전히 엉뚱한 위치에 수렴한 경우